

Разработка на сигурно чат приложение с End-to-End криптиране (E2EE)

1. Въведение и Цел на проекта

Да се разработи мрежово чат приложение (клиент-сървър архитектура) на езика C.

Приложението трябва да осигурява сигурна комуникация между двама потребители чрез End-to-End криптиране (E2EE), реализирано с помощта на криптографската библиотека OpenSSL.

Основно изискване: Сървърът да действа единствено като преносител на съобщенията (релейна станция) и в нито един момент да няма достъп до ключовете за декриптиране или до съдържанието на самите съобщения (plaintext).

2. Технически изисквания

2.1. Мрежова архитектура

- **Протокол:** TCP/IP.
- **Сървър:** Трябва да може да обслужва множество клиенти едновременно. Сървърът получава съобщение от Клиент А и го препраща към Клиент Б.
- **Клиент:** Конзолно приложение (CLI), което позволява едновременно въвеждане от клавиатурата и получаване на съобщения от мрежата.

2.2. Криптографски изисквания

Да се използва **хибридна криптосистема**, реализирана чрез библиотеката **OpenSSL**.

1. **Асиметрична криптография (Обмен на ключове):** * Използвайте RSA (минимум 2048-bit) или Elliptic Curve Diffie-Hellman (ECDH).
 - При свързване, клиентите трябва да си разменят публичните ключове през сървъра.
2. **Симетрична криптография (Криптиране на съобщенията):** * Използвайте **AES-256-GCM** (Galois/Counter Mode).
 - Всеки клиент генерира случаен сесийен ключ, криптира го с публичния ключ на отсрещната страна и го изпраща.
 - Тъй като GCM е AEAD (Authenticated Encryption with Associated Data) режим, той ще гарантира както конфиденциалността, така и целостта на съобщенията (чрез проверка на генерирания таг).
 - Всяко съобщение трябва да има уникален Initialization Vector (IV).
3. **Генериране на случайни числа:** За сесийните ключове и IV използвайте криптографски сигурни генератори (напр. `RAND_bytes()` от OpenSSL).

3. Фази на изпълнение (Препоръчителни стъпки)

- **Стъпка 1: Мрежова база.** Напишете TCP сървър и клиент. Направете така, че два клиента да могат да си чатят в чист текст (cleartext) през сървъра.
- **Стъпка 2: Интеграция на OpenSSL.** Добавете генериране на RSA/EC ключови двойки в клиента при стартиране. Напишете логика за сериализиране (PEM формат) и изпращане на публичния ключ до другия клиент.
- **Стъпка 3: Споделяне на сесията.** Генерирайте AES сесиен ключ, криптирайте го с получения публичен ключ и го изпратете. Декриптирайте го от другата страна с частния ключ.
- **Стъпка 4: Криптиран чат.** Заменете cleartext съобщенията от Стъпка 1 с AES-GCM криптирани пакети (съдържащи IV, Ciphertext и GCM Tag).

4. Изисквания за предаване

Проектът трябва да бъде предаден като линк към GitHub хранилище, съдържащ:

1. **Сорс код:** Всички `.c` и `.h` файлове.
2. **Makefile:** За автоматизирана компилация. Уверете се, че включва нужните флагове (напр. `-lcrypto -lssl -lpthread`).
3. **README.md:** Кратка документация, съдържаща:
 - Инструкции за компилиране и стартиране.
 - Описание на структурата на пакетите, които се изпращат по мрежата.
 - Описание на възникнали трудности (ако има такива).